EXPERIMENT - 3
CLI, GUI, AND VUI

AIM:
To develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) in Python.

TOOLS REQUIRED:
- Python 3.x
- VS Code
- Mac Operating System

THEORY:
The Smart Parking CLI System is a Python-based application that simulates parking slot management using both command-line and voice control. It tracks total, free, and occupied slots, updates them dynamically during live monitoring, and processes user commands through text or speech. The system uses speech recognition to understand voice input and text-to-speech to respond, demonstrating a basic AI-enabled smart parking solution.

CLI IMPLEMENTATION (Command Line Interface):

Code:

```
import random
import threading
import time
import speech_recognition as sr
import pyttsx3


class SmartParkingCLI:

   def __init__(self):
       self.total_slots = 50
       self.occupied_slots = 20
       self.free_slots = self.total_slots - self.occupied_slots
       self.running = True
       self.voice_mode = False
```

```python
        self.engine = pyttsx3.init()
        self.engine.setProperty('rate', 170)

    # ================= DISPLAY =================
    def dashboard(self):
        print("\n========== SMART PARKING DASHBOARD ==========")
        print(f"Total Slots     : {self.total_slots}")
        print(f"Free Slots      : {self.free_slots}")
        print(f"Occupied Slots  : {self.occupied_slots}")
        print("=============================================\n")

    def update_slots(self):
        change = random.choice([-1, 1])
        self.occupied_slots = max(0, min(self.total_slots,
                                         self.occupied_slots + change))
        self.free_slots = self.total_slots - self.occupied_slots

    def live_monitor(self):
        print("\nLive monitoring started (Press Ctrl+C to stop)\n")
        try:
            while True:
                self.update_slots()
                print(f"Free: {self.free_slots} | Occupied: {self.occupied_slots}")
                time.sleep(2)
        except KeyboardInterrupt:
            print("\nStopped live monitoring.\n")

    # ================= VOICE =================
    def speak(self, text):
        self.engine.say(text)
        self.engine.runAndWait()

    def voice_listener(self):
        recognizer = sr.Recognizer()
        mic = sr.Microphone()

        print("Voice mode activated. Say 'exit voice' to stop.")

        while self.voice_mode:
            try:
                with mic as source:
                    print("Listening...")
```

```
                recognizer.adjust_for_ambient_noise(source, duration=0.5)
                audio = recognizer.listen(source)

            command = recognizer.recognize_google(audio).lower()
            print("You said:", command)

            if "exit voice" in command:
                self.voice_mode = False
                print("Voice mode deactivated.")
                break

            self.process_command(command)

        except:
            print("Could not understand...")

# ================ COMMAND PROCESS ================
def process_command(self, command):

    if command == "dashboard":
        self.dashboard()

    elif command == "live":
        self.live_monitor()

    elif command == "free":
        print(f"Free slots: {self.free_slots}")
        self.speak(f"There are {self.free_slots} free slots")

    elif command == "occupied":
        print(f"Occupied slots: {self.occupied_slots}")
        self.speak(f"There are {self.occupied_slots} occupied slots")

    elif command == "status":
        print(f"Total: {self.total_slots} | Free: {self.free_slots} | Occupied:
{self.occupied_slots}")
        self.speak(f"Total {self.total_slots}, Free {self.free_slots}, Occupied
{self.occupied_slots}")

    elif command == "voice":
        self.voice_mode = True
        threading.Thread(target=self.voice_listener, daemon=True).start()
```

```python
        elif command == "help":
            print("""
Available Commands:
dashboard  - Show parking stats
live       - Start live monitor
free       - Show free slots
occupied   - Show occupied slots
status     - Show complete status
voice      - Activate voice mode
help       - Show commands
exit       - Exit program
""")

        elif command == "exit":
            self.running = False

        else:
            print("Unknown command. Type 'help'.")

    # ================= RUN =================
    def run(self):
        print("\n🚗 SMART PARKING SYSTEM - CLI VERSION")
        print("Type 'help' to see available commands.\n")

        while self.running:
            command = input(">> ").lower()
            self.process_command(command)

        print("System shutting down...")


if __name__ == "__main__":
    app = SmartParkingCLI()
    app.run()
```

Output:

```
>> dashboard

========== SMART PARKING DASHBOARD ==========
Total Slots      : 50
Free Slots       : 30
Occupied Slots   : 20
==============================================

>> free
Free slots: 30
```

How it Works:

1The system initializes with 50 total slots, 20 occupied, and calculates free slots automatically.

2 The user interacts through the Command Line Interface (CLI) by typing commands.

3 When a command is entered, the system checks it in the `process_command()` function.

4 The `dashboard` command displays total, free, and occupied slots.

5The `live` command starts real-time monitoring, where occupied slots increase or decrease randomly every 2 seconds.

6The `free`, `occupied`, and `status` commands display current parking statistics.

7 When `voice` mode is activated, the microphone listens to user speech and converts it to text using speech recognition.

8The system processes the spoken command and responds using text-to-speech, then continues running until the user types `exit`.

GUI IMPLEMENTATION (Graphic User Interface)

THEORY:

A Graphical User Interface (GUI) allows users to interact with a program using graphical elements such as buttons, labels, and text fields instead of typing commands.

Code:

```
import tkinter as tk
from tkinter import messagebox
from datetime import datetime
import random

class SmartParkingGUI:

    def __init__(self, root):
        self.root = root
        self.root.title("Smart Parking")
        self.root.geometry("800x500")

        self.total_slots = 10
        self.slots = [{"occupied": random.choice([True, False]), "vehicle": None}
                      for _ in range(self.total_slots)]
        self.vehicle_log = []

        self.create_dashboard()

    # ================= DASHBOARD =================
    def create_dashboard(self):
        tk.Label(self.root, text="SMART PARKING DASHBOARD",
                 font=("Arial", 18, "bold")).pack(pady=10)

        self.status_label = tk.Label(self.root, font=("Arial", 12))
        self.status_label.pack(pady=5)

        tk.Button(self.root, text="Refresh",
                  command=self.update_status).pack(pady=5)

        tk.Button(self.root, text="Register Entry",
                  command=lambda: self.register_vehicle("Entry")).pack(pady=5)
```

```python
    tk.Button(self.root, text="Register Exit",
              command=lambda: self.register_vehicle("Exit")).pack(pady=5)

    self.update_status()

# ================= STATUS =================
def update_status(self):
    occupied = sum(1 for s in self.slots if s["occupied"])
    free = self.total_slots - occupied
    self.status_label.config(
        text=f"Total: {self.total_slots} | Free: {free} | Occupied: {occupied}"
    )

# ================= ENTRY / EXIT =================
def register_vehicle(self, action):
    vehicle = f"TN-{random.randint(10,99)}-AB"
    time = datetime.now().strftime("%H:%M:%S")

    if action == "Entry":
        for slot in self.slots:
            if not slot["occupied"]:
                slot["occupied"] = True
                slot["vehicle"] = vehicle
                self.vehicle_log.append((vehicle, "Entry", time))
                messagebox.showinfo("Success", f"{vehicle} Entered")
                self.update_status()
                return
        messagebox.showwarning("Full", "No Free Slots")

    else:  # Exit
        for slot in self.slots:
            if slot["occupied"]:
                slot["occupied"] = False
                slot["vehicle"] = None
                self.vehicle_log.append((vehicle, "Exit", time))
                messagebox.showinfo("Success", f"{vehicle} Exited")
                self.update_status()
                return
        messagebox.showwarning("Empty", "No Vehicles Found")
```

```python
if __name__ == "__main__":
    root = tk.Tk()
    app = SmartParkingGUI(root)
    root.mainloop()
```

How it works:

1 The system initializes with a fixed number of parking slots (e.g., 10).

2 Each slot is randomly set as occupied or free at the start.

3 The dashboard displays total, free, and occupied slots.

4 When the **Refresh** button is clicked, slot statistics are recalculated and updated.

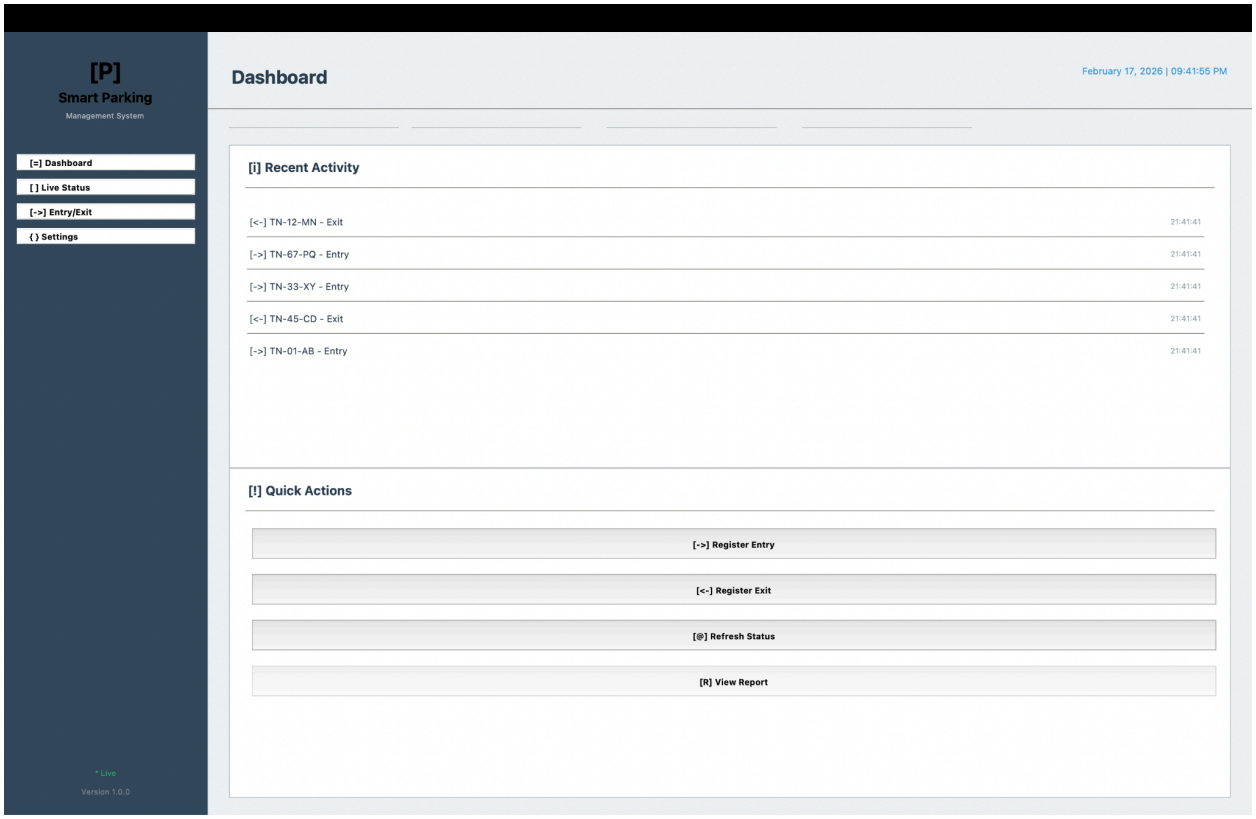
5 When **Register Entry** is clicked, the system finds a free slot and assigns a vehicle to it.

6 If no free slots are available, a warning message is shown.

7 When **Register Exit** is clicked, the system frees an occupied slot.

8 Every entry and exit action is recorded with vehicle number and time.

Output:



3.VUI IMPLEMENTATION(VOICE USER INTERFACE):

THEORY:

A Voice User Interface (VUI) allows users to interact with a system using voice commands instead of typing or clicking.

Code:

```
import tkinter as tk
from tkinter import ttk
from datetime import datetime
import random
import speech_recognition as sr
import pyttsx3
import threading
```

```python
class SmartParkingVUI:

    def __init__(self, root):
        self.root = root
        self.root.title("Smart Parking AI - Voice System")
        self.root.geometry("1100x650")
        self.root.resizable(False, False)

        # Parking Data
        self.total_slots = 50
        self.occupied_slots = 18
        self.free_slots = self.total_slots - self.occupied_slots

        # Voice System
        self.engine = pyttsx3.init()
        self.engine.setProperty('rate', 170)
        self.voice_active = False
        self.wake_word = "hey parking"

        self.create_layout()
        self.update_clock()

    # ================= UI DESIGN =================
    def create_layout(self):

        self.root.configure(bg="#0f172a")

        # Header
        self.header = tk.Frame(self.root, bg="#111827", height=80)
        self.header.pack(fill="x")

        tk.Label(self.header,
                 text="SMART PARKING AI",
                 bg="#111827",
                 fg="#00f5ff",
                 font=("Arial", 24, "bold")).pack(side="left", padx=30)

        self.clock_label = tk.Label(self.header,
                                    bg="#111827",
                                    fg="white",
```

```
                                font=("Arial", 16))
        self.clock_label.pack(side="right", padx=30)

        # Voice Buttons
        self.voice_btn = tk.Button(self.header,
                                   text="🎤 Activate Voice",
                                   bg="#22c55e",
                                   fg="white",
                                   font=("Arial", 12, "bold"),
                                   command=self.start_voice)
        self.voice_btn.pack(side="right", padx=10)

        self.stop_btn = tk.Button(self.header,
                                  text="🛑 Stop Voice",
                                  bg="#ef4444",
                                  fg="white",
                                  font=("Arial", 12, "bold"),
                                  command=self.stop_voice)
        self.stop_btn.pack(side="right")

        # Content
        self.content = tk.Frame(self.root, bg="#0f172a")
        self.content.pack(fill="both", expand=True)

        # Voice Status
        self.status_label = tk.Label(self.root,
                                     text="Voice: OFF",
                                     bg="#0f172a",
                                     fg="white",
                                     font=("Arial", 12))
        self.status_label.pack(pady=5)

        # Speech Display Area (FIXED POSITION)
        self.speech_frame = tk.Frame(self.root, bg="#0f172a")
        self.speech_frame.pack(pady=10)

        tk.Label(self.speech_frame,
                 text="🎙 You Said:",
                 bg="#0f172a",
                 fg="#00f5ff",
                 font=("Arial", 14, "bold")).pack()
```

```
        self.speech_text = tk.Label(self.speech_frame,
                                    text="...",
                                    bg="#1e293b",
                                    fg="white",
                                    font=("Arial", 14),
                                    width=60,
                                    height=2)
        self.speech_text.pack(pady=5)

        self.show_dashboard()

    # ================= DASHBOARD =================
    def show_dashboard(self):
        self.clear_content()

        self.create_card("TOTAL SLOTS", self.total_slots, "#3b82f6", 200)
        self.create_card("FREE SLOTS", self.free_slots, "#22c55e", 450)
        self.create_card("OCCUPIED", self.occupied_slots, "#ef4444", 700)

        self.progress = ttk.Progressbar(self.content,
                                        length=600,
                                        mode="determinate")
        self.progress.place(x=250, y=400)

        self.update_progress()

    def create_card(self, title, value, color, x_pos):
        card = tk.Frame(self.content,
                        bg="#1e293b",
                        width=220,
                        height=150,
                        highlightbackground=color,
                        highlightthickness=3)
        card.place(x=x_pos, y=150)

        tk.Label(card,
                 text=title,
                 bg="#1e293b",
                 fg="white",
                 font=("Arial", 14)).pack(pady=10)

        tk.Label(card,
```

```
                text=value,
                bg="#1e293b",
                fg=color,
                font=("Arial", 32, "bold")).pack()

# ================= LIVE =================
def show_live(self):
    self.clear_content()

    tk.Label(self.content,
                text="LIVE MONITORING ACTIVE",
                bg="#0f172a",
                fg="#00f5ff",
                font=("Arial", 22, "bold")).pack(pady=30)

    self.live_status = tk.Label(self.content,
                                bg="#0f172a",
                                fg="white",
                                font=("Arial", 18))
    self.live_status.pack(pady=20)

    self.simulate_live()

def simulate_live(self):
    if not hasattr(self, "live_status"):
        return

    change = random.choice([-1, 1])
    self.occupied_slots = max(0, min(self.total_slots,
                                     self.occupied_slots + change))
    self.free_slots = self.total_slots - self.occupied_slots

    self.live_status.config(
        text=f"Free: {self.free_slots} | Occupied: {self.occupied_slots}")

    self.update_progress()
    self.root.after(2000, self.simulate_live)

# ================= VOICE CONTROL =================
def start_voice(self):
    if not self.voice_active:
        self.voice_active = True
```

```
        self.status_label.config(text="Voice: ON", fg="#22c55e")
        threading.Thread(target=self.voice_listener, daemon=True).start()
        self.speak("Voice system activated")

    def stop_voice(self):
        self.voice_active = False
        self.status_label.config(text="Voice: OFF", fg="white")
        self.speak("Voice system deactivated")

    def speak(self, text):
        self.engine.say(text)
        self.engine.runAndWait()

    def voice_listener(self):
        recognizer = sr.Recognizer()
        mic = sr.Microphone()

        while self.voice_active:
            try:
                with mic as source:
                    self.speech_text.config(text="Listening...")
                    recognizer.adjust_for_ambient_noise(source)
                    audio = recognizer.listen(source)

                command = recognizer.recognize_google(audio).lower()

                self.speech_text.config(text=command)

                if self.wake_word in command:
                    self.process_command(command)

            except:
                self.speech_text.config(text="Could not understand...")
                continue

    def process_command(self, command):

        if "dashboard" in command:
            self.show_dashboard()
            self.speak("Opening dashboard")

        elif "live" in command:
```

```
            self.show_live()
            self.speak("Live monitoring started")

        elif "free" in command:
            self.speak(f"There are {self.free_slots} free slots")

        elif "occupied" in command:
            self.speak(f"There are {self.occupied_slots} occupied slots")

        elif "exit" in command:
            self.speak("Shutting down system")
            self.root.quit()

    # ================= UTILITIES =================
    def update_progress(self):
        percent = (self.occupied_slots / self.total_slots) * 100
        if hasattr(self, "progress"):
            self.progress["value"] = percent

    def update_clock(self):
        now = datetime.now().strftime("%H:%M:%S")
        self.clock_label.config(text=now)
        self.root.after(1000, self.update_clock)

    def clear_content(self):
        for widget in self.content.winfo_children():
            widget.destroy()


if __name__ == "__main__":
    root = tk.Tk()
    app = SmartParkingVUI(root)
    root.mainloop()
```

How it Works:

1 The system initializes with **50 total parking slots**, 18 occupied slots, and calculates free slots automatically.

2 The GUI dashboard displays **Total, Free, and Occupied slots** using styled cards and a progress bar.

3 The progress bar shows the **percentage of parking utilization** based on occupied slots.

4 The clock in the header updates every second using the `update_clock()` function.

5 When **Live Monitoring** is activated, the system randomly increases or decreases occupied slots every 2 seconds to simulate real-time parking activity.

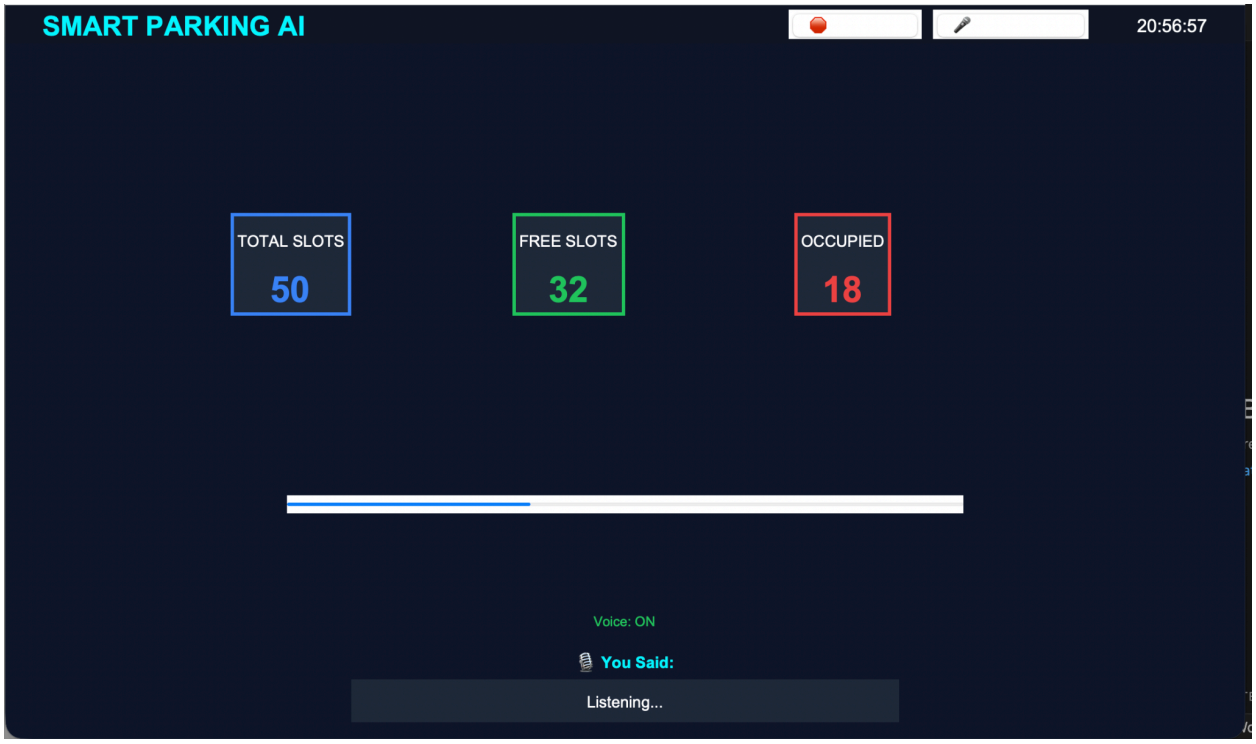
6 When the **Activate Voice** button is clicked, a background thread starts listening for voice commands using the microphone.

7 The system listens for a **wake word ("hey parking")** before processing any command.

8 Recognized voice commands like “dashboard”, “live”, “free”, “occupied”, or “exit” trigger corresponding actions in the GUI.

9 The system responds using **text-to-speech**, providing spoken feedback while updating the interface visually.

Output:



RESULT:

The CLI, GUI, and VUI versions of the program were successfully implemented and tested. All interfaces performed the required operations correctly. The CLI allowed text-based interaction, the GUI provided a user-friendly graphical interface, and the VUI enabled voice-based interaction. Each interface demonstrated proper functionality and error handling.